

EVALUATION FORM

GAP AI Impact Evaluation

● Processing

GAP AI Impact Evaluation

This form measures your real, demonstrated AI impact across the last six months. We are not measuring familiarity with AI tools — we are measuring outcomes, judgment, and the structural changes you have made to your work because of AI.

Please answer each section in detail. Where useful, attach artifacts (screenshots, snippets, links, PDFs, markdown notes) that enrich the evidence for your response. Quality of evidence carries weight; vague claims do not.

What we are looking for

- **Daily AI-first workflow.** How AI is woven into the cadence of your work, not used as an occasional helper.
- **AI output auditing and validation.** How you verify what AI produces before committing or shipping it.
- **AI-driven value and metrics.** Concrete metrics that show your AI usage moved the needle — for clients, for the team, or for the organization.
- **AI agents and automation.** Whether you are building, orchestrating, or maintaining autonomous AI workflows beyond single-shot prompts.

- **Extending AI value and leadership.** How you scale your AI impact across your team or the company — knowledge sharing, mentoring, system design.

How to fill this out

- Be specific. Cite tools, projects, numbers, and dates.
- Attach artifacts when they materially strengthen a claim.
- The optional "artifacts description" field lets you explain the relevance of the files you attach.
- You may save and return to this form anytime while the cycle is open. Drafts persist automatically.

When you are ready, click **Submit for processing** at the bottom. Once submitted, the AI evaluator will review your responses — you will not be able to edit them after that point.

YOUR RESPONSES · 6 questions

COLLAPSE ALL

1 Project Constraints

RESPONSE

- Security or compliance restrictions
- Highly regulated environment
- Client limitations on AI usage

2 Describe your daily AI-first workflow: How specifically do you integrate AI into your task planning, ...

RESPONSE

Over the past six months, I have shifted from using AI as an occasional productivity aid to integrating it as the primary tool for producing project-level deliverables. The following describes how that integration works in practice.

My repeatable AI-first workflow pattern:

When I receive a new task — whether it involves test design, documentation, analysis, or data generation — my first step is to frame the task for AI assistance before doing any manual work. I provide the AI with source documents or requirements as structured context, specify constraints and expected output format, and iterate across multiple prompts to refine, validate, and reshape the output until it meets delivery standards. Only then do I apply a final human review before committing the artifact.

Applied examples from the last six months:

Test data generation (Artifact B): When the project required synthetic SQL test data for CustomerActivityLogs and CustomerExceptionLogs, I used GitHub Copilot to design deterministic INSERT scripts covering six months of realistic records — including action types, status distributions, and timestamping logic. This replaced what would have been one to two days of manual scripting with approximately one hour of guided AI iteration. The scripts were used directly in project testing.

Performance Test strategy documentation (Artifacts C): I used AI to draft a performance QA strategy for the DOM project. This was reviewed by Project team managers and delivered to the client for review — a deliverable that would have taken three to five days manually was completed in approximately one day.

Requirements analysis (Artifact A): I used AI to compare the MediQuant Requirements document against the original GAP proposal, convert informal notes into management-ready findings, and produce two official deliverable documents. These were shared with the client and team as formal project outputs.

Test case design and ADO import (Artifact D): I used AI to generate twenty-seven new test cases requested by the client after reviewing the Test Plan, then prompted AI to reformat them as an ADO-compatible CSV import. The cases were imported directly into Azure DevOps and are in active use.

Meeting documentation (Artifact F): After the May 26 performance testing review, I used Microsoft Copilot to transcribe and structure the 66-minute meeting into formal minutes — capturing decisions, risks, open questions, and action items — in approximately 15 minutes.

Project constraints: The client operates in a highly regulated environment with restrictions on AI usage

to prevent PII exposure. All AI work is performed exclusively on non-sensitive artifacts: documentation, synthetic data, test strategies, and analysis documents. No client data, production systems, or PII-adjacent content enters AI tools.

Progression from April: In April, my AI usage produced personal productivity gains but no project-level deliverables. By July, every artifact above was officially adopted — shared with the team, presented to client stakeholders, or imported into project systems — marking a clear transition from Level 1 to Level 2 impact.

ARTIFACTS

↓ B CustomerActivityAndExceptionLogs sql scripts creation.md	4.4 KB
↓ F Performance_Testing_Meeting_Minutes.md	4.1 KB
↓ A Client requirements vs proposal comparizon.md	2.4 KB
↓ D DOM_QA_Strategy_New_Requested_TestCases creation and json for ADO.md	12.1 KB
↓ C dom_performance_testing_strategy html.pdf	2.8 MB

3 Describe how you audit AI outputs before committing them. How do you stress-test AI-generated s... ^

RESPONSE

Validating AI-generated outputs is a deliberate step in my workflow, not an afterthought. Before any AI-produced artifact is committed or shared, I apply the following review process:

Edge case testing:
For code and scripts, I manually trace through boundary conditions before accepting output. When I used GitHub Copilot to generate the CustomerActivityLogs and CustomerExceptionLogs SQL scripts (Artifact B), I verified timestamp distribution logic, status probability ratios, and seed determinism by dry-running the scripts against a test schema and inspecting the resulting records. I also confirmed that the scripts produced the expected six-month range and that edge cases — such as month boundaries and end-of-month day calculations — were handled correctly.

For the new requested test cases (Artifact D), I deliberately included negative, mixed, and boundary scenarios (missing files, malformed paths, no file-path columns detected) as part of the AI-generated test set — treating edge case coverage as a design requirement fed into the prompt, not as a post-review addition.

Security review:

My primary governance control is prompt hygiene: no PII, client production data, or confidential identifiers are ever entered into AI tools. All synthetic data uses fixed placeholder values (e.g., jcasal@mediquant.com, CustomerId: 647) rather than real user data. AI-generated code is reviewed for unintended data exposure, hardcoded secrets, or unsafe patterns before use.

Architectural compliance:

I do not currently perform formal architectural compliance validation on AI outputs — this remains a gap, consistent with what I reported in April. In practice, I compensate by cross-referencing AI-generated content against existing project documentation, the QA strategy baseline (Artifact G), and known system constraints before accepting any architectural claims or assumptions in AI-produced documents.

Hallucination detection:

For analysis documents such as the requirements comparison (Artifact A), I validated every AI-produced finding against the source documents directly — confirming that identified gaps, misalignments, and Phase 2 flags were traceable to specific sections of the MediQuant Requirements document and the original GAP proposal. Any AI claim that could not be verified against source material was removed or flagged for clarification.

ARTIFACTS

- | | |
|--|--------|
| ↓ G DOM Testing Strategy document.pdf | 3.5 MB |
| ↓ B CustomerActivityAndExceptionLogs sql scripts creation.md | 4.4 KB |
| ↓ A Client requirements vs proposal comparizon.md | 2.4 KB |

4 What data illustrates your AI usage is driving value? Cite specific metrics, not estimates, such as v... ^

RESPONSE

The following table summarizes conservative time-savings estimates across the six AI-assisted deliverables produced between April and July 2026. All estimates are based on direct experience with the tasks involved and reflect realistic manual effort for a QA engineer working without AI assistance.

Artifact Deliverable	Manual Estimate	With AI	Hours Saved
A Requirements vs Proposal Comparison + Management Report	3–4 days	~4 hours	~20 hours
B SQL Test Data Scripts (2 tables, 6-month range)	1–2 days	~1 hour	~7 hours
C Performance Test Plan	3–5 days	~1 day	~16 hours
D Unstructured Test Cases + ADO CSV Import (7 cases)	1 day	~1.5 hours	~6.5 hours
E Performance Testing Plan Annex 1	2–3 days	~half day	~12 hours
F Performance Testing Meeting Minutes	~2–3 hours	~15 minutes	~2 hours
Total (conservative)			~63.5 hours (~8 working days)

Nature of the impact:

These are not trivial productivity gains on individual tasks. Every artifact in this table was officially adopted: the Performance QA strategy became the project testing baseline, the performance testing plan was presented to client stakeholders and used as the official starting point, the test cases were imported into Azure DevOps and are actively executed, and the requirements comparison was delivered as a formal management report. The time saved directly translated into faster project delivery and higher-quality outputs than would have been achievable manually within the same timeframes.

Contrast with April baseline:

In April, my reported AI productivity gains were personal — faster documentation and individual task speed — with no project-level adoption. By July, every deliverable above was shared with the team, the client, or integrated into project systems, representing a clear shift from individual efficiency to measurable project-level impact.

Bottleneck flagged for optimization:

The May 26 performance testing review (Artifact F) surfaced a significant manual bottleneck: workload characterization and dataset creation. Defining a realistic test baseline currently requires manual analysis of production data profiles, client-project breakdowns, and peak-load scenarios — a process that involves significant back-and-forth with stakeholders and manual data interpretation. This is a strong candidate for AI-assisted automation: structured prompt workflows could accelerate workload modeling, dataset generation, and scenario documentation significantly.

ARTIFACTS

↓ A Client requirements vs proposal comparizon.md	2.4 KB
↓ B CustomerActivityAndExceptionLogs sql scripts creation.md	4.4 KB
↓ D DOM_QA_Strategy_New_Requested_TestCases creation and json for ADO.md	12.1 KB
↓ C dom_performance_testing_strategy.html.pdf	2.8 MB
↓ F Performance_Testing_Meeting_Minutes.md	4.1 KB

5 How do you use AI agents, automated pipelines, workflows, or customized tools to solve complex p... ^

RESPONSE

I do not currently use formal AI agents, automated pipelines, RAG implementations, fine-tuned models, or fallback mechanisms. I want to be direct about this rather than overstate my position.

What I do use: structured multi-step conversational orchestration

While not a formal agentic pipeline, my workflow follows a deliberate sequence that goes beyond single-prompt interaction. For each major deliverable, I apply a consistent pattern:

Context loading — I provide source documents, requirements, or prior artifacts as structured input, giving the AI a grounded knowledge base for the task rather than relying on general knowledge.
Staged prompt chaining — I break complex deliverables into sequential prompts: first generating a

structural outline, then expanding each section, then reformatting for the target audience or system (e.g., converting test case descriptions into ADO-compatible CSV in Artifact D, or converting informal reviewer notes into management-facing language in Artifact A).

Constraint specification — Each prompt includes explicit output constraints: tone, format, what to exclude, and what to flag for human review.

Iterative refinement — Outputs are revised across multiple turns before I accept them as final, treating the AI as a collaborator rather than a one-shot generator.

The HTML deliverables (Artifacts C and E) are the clearest examples of this: both required multi-session, multi-stage AI interaction to produce structured, professionally formatted documents adopted by the project.

Why formal agents have not been feasible:

The client's regulated environment and AI usage restrictions make automated pipelines that process project data impractical and non-compliant at this stage.

Next steps toward Level 3:

I have already completed GAP's internal Autonomous Engineer training, which provided a structured foundation in agentic patterns, orchestration concepts, and multi-step AI workflow design. Building on that, I am currently pursuing advanced external training — including Anthropic's recommended courses and selected Udemy programs — to deepen my practical agentic capabilities. My immediate near-term goal is to build a structured prompt library — reusable, parameterized prompts for recurring QA tasks — as a first step toward systematic AI workflow automation within the project's compliance constraints.

ARTIFACTS

- | | |
|---|----------|
| ↓ Autonomous_Engineer_Intensive_Training_-_Certificate.pdf | 838.9 KB |
| ↓ E Anex_1_Performance_Testing_Plan_Peak_Month_Capacity_Simulation.html.pdf | 944.2 KB |
| ↓ C dom_performance_testing_strategy.html.pdf | 2.8 MB |

6 Describe any actions you've taken to extend AI value beyond your own work. This could include: co... ^

RESPONSE

I have not formally mentored peers on AI practices or led an internal AI initiative during this cycle. I want to be transparent about that rather than overstate it.

Where scaling has occurred indirectly:

The most tangible way my AI usage has extended beyond my own work is through the deliverables themselves. The DOM QA Strategy (Artifact G) is now the team's working testing baseline, produced in a fraction of the time it would have required manually. The Performance Testing Plan (Artifact C) was presented to an internal review with the client-side stakeholders and adopted as the official testing baseline, directly influencing project direction. The requirements comparison (Artifact A) was delivered as a management-level report used in client-facing discussions. In each case, AI-produced work fed into team and client decisions — that is AI ROI made visible beyond the individual contributor level.

Near-term actions toward broader impact:

Having completed GAP's internal Autonomous Engineer training, I have a structured foundation to

share. My next steps will include contributing to the current template for generating QA testing



WORKSPACE

 Dashboard

Dashboard

AI proficiency evaluations

EMPLOYEE



Jose Mora Casal ▾

available to support any internal initiative that benefits from a QA or testing perspective on AI-assisted delivery.

ARTIFACTS

 C dom_performance_testing_strategy.html.pdf	2.8 MB
 G DOM Testing Strategy document.pdf	3.5 MB
 A Client requirements vs proposal comparizon.md	2.4 KB